Clarifying the annelid tree of life amid deep phylogenomic conflict

Analysis command archive

Maeva Perez, Yunlong Li, Fan Hui, Yi-Xuan Li, and Yuanning Li
José M. Martín-Durán, Greg W. Rouse, Pei-Yuan Qian, and Jian-Wen Qiu

Contents

1 Scope and use **2**
1.1 Languages and path convention . . . 2

2 Assembling transcriptomes **2**

3 Merging transcriptomes **2**

4 Finding gene families (orthogroups) **4**

5 Splitting gene families into single-copy orthologs **4**

6 Getting synteny-informed orthologs **5**

7 Computing gene trees **6**
7.1 Realigning and trimming . . . 6
7.2 Computing OG trees (LG/WAG + tree) . . . 7
7.3 Computing SOG trees (site-homogeneous model selection + tree) . . . 7
7.3.1 Computing trees under a site-heterogeneous model . . . 8
7.4 Computing gene trees under a site-heterogeneous model . . . 9
7.5 Tree statistics and submatrices . . . 10

8 Computing species trees **11**
8.1 Concatenation approach . . . 11
8.1.1 Edge-proportional partitioned model . . . 11
8.1.2 Site-heterogeneous model . . . 11
8.2 Reconciliation approach . . . 11
8.3 Computing constrained trees . . . 12

9 Assessing node support **13**
9.1 For species trees . . . 13
9.2 For constrained trees . . . 14
9.3 PhyTop . . . 16

10 Incomplete lineage sorting and introgression **16**

11 Hypothesis testing **16**
11.1 SH and AU tests . . . 16
11.2 Likelihood mapping . . . 16
11.3 DeltaL analysis . . . 17

12 Time calibration **18**
12.1 Get calibrations . . . 18
12.2 Run time tree . . . 18

12.3 Assess convergence and parse results 20

1 Scope and use

This document records representative commands used in the study. It is provided for methodological transparency and is not a turnkey reproducibility workflow. The analyses were run on a computing server, and the directory structure, environment names, software locations, and filenames shown below reflect that working environment. Repository filenames have subsequently been standardized for clarity, so commands and paths are not expected to run unchanged after cloning the repository.

1.1 Languages and path convention

Every code chunk declares its language explicitly. Bash is used for command-line programs and workflow orchestration, and R for the CladeDate analysis. Complete Python data-processing and plotting programs are provided as standalone scripts or notebooks. All chunks are intentionally marked `eval=FALSE`.

The original analyses used absolute paths on a computing server and a local workstation. Those personal paths have been removed. In the Bash excerpts, `ANALYSIS_ROOT` denotes the root of the **original analysis workspace**, not the root of this GitHub repository:

```
# Language: Bash
export ANALYSIS_ROOT="/path/to/original_analysis_workspace"
export REPOSITORY_ROOT="/path/to/Annelida_phylogenomics"
export DATA_DIR="${ANALYSIS_ROOT}/data"
export SCRATCH_DIR="${ANALYSIS_ROOT}/scratch"
export BUSCO_LINEAGE="/path/to/busco/metazoa_odb10"
export EVIDENTIALGENE_HOME="/path/to/EvidentialGene"
export PARGENES_HOME="/path/to/pargenes"
```

Set these variables before adapting a Bash excerpt. Using an explicit root is safer than replacing an absolute prefix with `./`, because `./` changes meaning after a command changes the working directory. Programs and environment managers are assumed to be available on `PATH`; installation-specific executable locations are deliberately omitted.

2 Assembling transcriptomes

```
# Language: Bash
## Assembling raw reads

Trinity --seqType fq --max_memory 50G --left $R1 --right $R2 --CPU 16 --output trinity_output

TransDecoder.LongOrfs -t $fasta
TransDecoder.Predict -t $fasta --single_best_only --cpu 4

## QC

busco -i "$fasta" -l "$BUSCO_LINEAGE" -o "busco_${fasta}" -m prot -c 1 -f
```

3 Merging transcriptomes

```
# Language: Bash
species=`awk -F'\t' -v species_index="$i" '{if($1==species_index) print $2}'
filelist_species.tsv`

DATADIR="${DATA_DIR}/transcriptomic_assemblies/${species}"

filelist=filelist_transcripto_all.tsv

assemblies=`awk -F"\t" -vspecies="$species" -vDATADIR="$DATADIR" -vOFS="/" '{if ($7==species)
print DATADIR,$5}' $filelist`  #| awk '{ORS=(NR%99?FS:RS)}1' `

echo "RUNNING SPECIES $i: $species"
echo "ASSEMBLIES= $assemblies"

## merge inputfiles

rm -rf "${SCRATCH_DIR}/trans_assembly/${species}/tr2aacds_3"

mkdir -p "${SCRATCH_DIR}/trans_assembly/${species}/tr2aacds_3"
cd "${SCRATCH_DIR}/trans_assembly/${species}/tr2aacds_3"

cat $assemblies | sed 's/|/ /g' > tmp.transcripts.fasta

"${EVIDENTIALGENE_HOME}/scripts/rnaseq/trformat.pl" -output transcripts.fasta -input
tmp.transcripts.fasta

## run tr2aacds

"${EVIDENTIALGENE_HOME}/scripts/prot/tr2aacds.pl" -cdnaseq transcripts.fasta -logfile -runsteps
-noutrorf


## extract unique

cp okayset/transcripts.okay.aa ../${species}.merged.transcripts.okay.aa
cp okayset/transcripts.okay.cds ../${species}.merged.transcripts.okay.cds
cp okayset/transcripts.okay.mrna ../${species}.merged.transcripts.okay.mrna


seqkit grep -r -n -p 't1 ' okayset/transcripts.okay.aa >
../${species}.merged.transcripts.okay.unique.aa

## run busco

mamba activate busco_v550

busco -i ../${species}.merged.transcripts.okay.aa -l "$BUSCO_LINEAGE" -o
busco_transcripts.okay.aa -m prot -c 1 -f
cp busco_transcripts.okay.aa/run_metazoa_odb10/short_summary.txt
../${species}.merged.transcripts.okay.aa.busco

busco -i ../${species}.merged.transcripts.okay.unique.aa -l "$BUSCO_LINEAGE" -o
busco_transcripts.okay.unique.aa -m prot -c 1 -f
cp busco_transcripts.okay.unique.aa/run_metazoa_odb10/short_summary.txt
../${species}.merged.transcripts.okay.unique.aa.busco
```

```
id=80
cd-hit -i ../${species}.merged.transcripts.okay.unique.aa -o
../${species}.merged.transcripts.okay.unique.cdhit${id}.pep -c 0.${id} -M 4000 -T 1

busco -i ../${species}.merged.transcripts.okay.unique.cdhit${id}.pep -l "$BUSCO_LINEAGE" -o
busco_transcripts.okay.unique.cdhit${id}.pep -m prot -c 1 -f
cp busco_transcripts.okay.unique.cdhit${id}.pep/run_metazoa_odb10/short_summary.txt
../${species}.merged.transcripts.okay.unique.cdhit${id}.pep.busco


## get summary of buscos

if [[ -f ../${species}.merged.transcripts.okay.unique.aa ]] && [[ -f
../${species}.merged.transcripts.okay.unique.aa ]]; then

echo -e "SUCCESS\t${i}\t${species}\t$(date '+%Y-%m-%d')\t$(grep 'C:'
../${species}.merged.transcripts.okay.unique.cdhit${id}.pep.busco | sed 's/\t//g')"  >>
work/Trans_assembly/summary_tr2aacds_processed.txt

else

echo -e "FAIL\t${i}\t${species}\t$(date '+%Y-%m-%d')\tNA" >>
work/Trans_assembly/summary_tr2aacds_processed.txt

fi
```

4 Finding gene families (orthogroups)

```
# Language: Bash

# Finding orthogroups with Orthofinder

datadir=144species #A31 for reduced dataset

orthofinder -f $datadir -t $SLURM_CPUS_PER_TASK -a $SLURM_CPUS_PER_TASK -S diamond -T fasttree
-ot -I 1.5 -M msa -z
```

5 Splitting gene families into single-copy orthologs

```
# Language: Bash

CPUS=$1

mamba activate orthosnap
export PERL5LIB= PERL_LOCAL_LIB_ROOT= cpan

mkdir -p "${ANALYSIS_ROOT}/OrthoFinder/Orthosnap"
cd "${ANALYSIS_ROOT}/OrthoFinder/Orthosnap"

orthosnap () {
```

```
OG=$1

mkdir -p snaps
mkdir -p out

orthosnap -t input_trees/${OG}.tree -f input_aln/${OG}.fa -op snaps/ -o 4 > out/${OG}.out


##### summary of the results

if [ -s snaps/${OG}.fa.orthosnap.0.fa ]; then

echo -e "$OG\tsnapped" >> OG_orthosnap_result.txt

else
if [ "$(tail -n2 out/$OG.out | head -n1)" == "Input phylogeny is already a single-copy
orthogroup" ]; then

echo -e "$OG\tnot" >> OG_orthosnap_result.txt
ln -s aln/${OG}.fa snaps/${OG}.fa # make a soft copy to the snaps folder

else
echo -e "$OG\terror" >> OG_orthosnap_result.txt
fi
fi

}

export -f orthosnap


parallel -j $CPUS orthosnap {1} :::: OG.filelist
```

6 Getting synteny-informed orthologs

Complete custom scripts used for the synteny analyses are provided separately: `Scripts/Synteny/identify_microsynteny.py` identifies pairwise collinear blocks from BED and mutual-best-hit files, and `Scripts/Synteny/shared_fusion_plot.R` builds the shared chromosomal-fusion matrix and heatmap. Representative invocations are:

```
# Language: Bash
python "${REPOSITORY_ROOT}/Scripts/Synteny/identify_microsynteny.py" \
  --bed1 species_1.bed \
  --bed2 species_2.bed \
  --mbh species_1_species_2.mbh \
  --output species_1_species_2.blocks.tsv \
  --simple species_1_species_2.blocks.simple.tsv

Rscript "${REPOSITORY_ROOT}/Scripts/Synteny/shared_fusion_plot.R" \
  shared.fusion.Annelida.txt \
  ordered_intersection_shared_fusions.Annelida.pdf \
  ordered_intersection_shared_fusions_matrix.Annelida.csv
```

7 Computing gene trees

7.1 Realigning and trimming

```
# Language: Bash
mkdir -p trimmed
mkdir -p trimal_out
mkdir -p phykit

trim () {

file=$1

### trim
## remove columns with > 90% gaps
## residue must be present in >50% of sequences and at least 30% of residues must pass the
threshold

trimal -in ../Orthosnap/input_aln/${file}.fa -out trimmed/${file}.faa -keepheader -fasta -gt
0.1 -resoverlap 0.5 -seqoverlap 30 > trimal_out/${file}.out

if [[ ! -s "trimal_out/${file}.out" ]]; then
rm trimal_out/${file}.out
fi

if [[ -s "trimmed/${file}.faa" ]]; then

echo $file | awk -vOFS="\t" '{print $1,"ok"}' >> trimmal_results.txt

## phykit1
tax_occ=`grep -E "^>[0-9]+|[0-9]+$" trimmed/${file}.faa | awk -F"|" '{print $1}' | sort | uniq
| wc -l`
seqcount=`grep ">" trimmed/${file}.faa | wc -l`

echo -e "$file\t$tax_occ\t$seqcount" >> phykit/tax_occ.tsv

pk_rcv trimmed/${file}.faa | awk -F'\t' -vOFS='\t' -valn="$file" '{print aln,$0}' >>
phykit/pk_rcv.tsv

pk_pis trimmed/${file}.faa | awk -F'\t' -vOFS='\t' -valn="$file" '{print aln,$0}' >>
phykit/pk_pis.tsv

pk_aln_len_no_gaps trimmed/${file}.faa | awk -F'\t' -vOFS='\t' -valn="$file" '{print
aln,$1,$2,$3}' >> phykit/pk_aln_len_no_gaps.tsv


else
echo $file | awk -vOFS="\t" '{print $1,"error"}' >> trimmal_results.txt
fi


}

export -f trim

parallel --jobs $CPUS trim {1} :::: OG_genes.list
```

```
parallel --jobs $CPUS trim {1} :::: SOG_genes.list
```

7.2 Computing OG trees (LG/WAG + tree)

```
# Language: Bash
run_fastree_LG () {
SUPERMATRIX=$1
ALIGNMENT=$2
OG=`echo $ALIGNMENT | awk -F'[/|.]' '{print $2}'`
TREE=${SUPERMATRIX}_trees/${OG}.tree

FastTree -lg -gamma -pseudo $ALIGNMENT > $TREE


if [ -s "$TREE" ] && [ -s "$ALIGNMENT" ]; then
echo '' | awk -vOFS="\t" -vOG="$OG" '{print "success",OG}' >> remaining_alignments.txt
else
echo '' | awk -vOFS="\t" -vOG="$OG" '{print "fail",OG}' >> remaining_alignments.txt
fi

}


run_fastree_wag () {
SUPERMATRIX=$1
ALIGNMENT=$2
OG=`echo $ALIGNMENT | awk -F'[/|.]' '{print $2}'`
TREE=${SUPERMATRIX}_trees/${OG}.tree

FastTree -wag -gamma -pseudo $ALIGNMENT > $TREE


if [ -s "$TREE" ] && [ -s "$ALIGNMENT" ]; then
echo '' | awk -vOFS="\t" -vOG="$OG" '{print "success",OG}' >> remaining_alignments.txt
else
echo '' | awk -vOFS="\t" -vOG="$OG" '{print "fail",OG}' >> remaining_alignments.txt
fi

}


export -f run_fastree_lg
export -f run_fastree_wag

parallel --jobs $CPUS run_fastree_lg "$SUPERMATRIX" {} :::: OG_genes.list


parallel --jobs $CPUS run_fastree_wag "$SUPERMATRIX" {} :::: large_OG_genes.list
```

7.3 Computing SOG trees (site-homogeneous model selection + tree)

```
# Language: Bash

pargenes.py -c $CPUS --alignments-dir ${SUPERMATRIX}_data --output-dir ${SUPERMATRIX}_working
-d aa -m --bs-trees 100 --parsimony-starting-trees 5 --random-starting-trees 5 --continue


# Extract trees and gene models

python "${PARGENES_HOME}/pargenes_src/export.py" -o ${SUPERMATRIX}_results -i
${SUPERMATRIX}_working --support-values-tree --best-ml-model

cd ${SUPERMATRIX}_results


for file in `find . -name '*.raxml.bestModel'`
do

OG=`echo $file | awk -F'[./]' '{print $3}' | sed 's/_/./g'`
sed "s/noname/$OG/g" $file >> ../${SUPERMATRIX}.bestModels

TREE=$(echo $file | sed 's/.raxml.bestModel/.support.raxml.support/g')
cat $TREE >> ../${SUPERMATRIX}.genetrees.nx

echo "${OG}" >> ../${SUPERMATRIX}.genetrees.nx.ids

done
```

7.3.1 Computing trees under a site-heterogeneous model

```
# Language: Bash
mkdir -p C60_trees

##########################################

# -mem option does not work with partition models yet

run_C60 () {

OG_aln=$1
OG2=$(echo "${OG_aln}" | awk -F'.' '{print $1}')

tree_file="${SCRATCH_DIR}/tree_building/OG75/LG_coalescent/OG75_trees/${OG2}.tree"
aln_file="${SCRATCH_DIR}/tree_building/OG75/OG75_data/${OG_aln}"
output_file=C60_trees/${OG2}.treefile

#echo $tree_file, $aln_file, $output_file

# Check if the output file already exists
if [ -f "$output_file" ]; then
    echo "$OG2 already processed, skipping."
    echo "$OG2" >> finished_processing.txt
    exit 0
fi
```

```
# Run iqtree2 if the output file does not exist
iqtree2 -s "$aln_file" -m LG+C60+F+G -ft "$tree_file" --seqtype AA --prefix C60_trees/${OG2}
-ntmax 1 -nt 1

# Check if iqtree2 ran successfully and the output file was created
if [ -f "$output_file" ]; then
    echo "$OG2" >> finished_processing.txt
else
    echo "$OG2" >> error_processing.txt
fi

}

export -f run_C60

ls -lS "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/OG75_data/" | awk '{print $NF}' | tail -n+2
| tac > OG75.size_sorted.filelist

parallel --jobs $CPUS run_C60 {} :::: OG_genes.list
parallel --jobs $CPUS run_C60 {} :::: SOG_genes.list
```

7.4 Computing gene trees under a site-heterogeneous model

```
# Language: Bash
mkdir -p C60_trees

run_C60 () {

OG_aln=$1
OG2=$(echo "${OG_aln}" | awk -F'.' '{print $1}')

tree_file=${OG2}.tree
aln_file=${OG_aln}
output_file=C60_trees/${OG2}.treefile

# Check if the output file already exists
if [ -f "$output_file" ]; then
    echo "$OG2 already processed, skipping."
    echo "$OG2" >> finished_processing.txt
    exit 0
fi

# Run iqtree2 if the output file does not exist
iqtree2 -s "$aln_file" -m LG+C60+F+G -ft "$tree_file" --seqtype AA --prefix C60_trees/${OG2}
-ntmax 1 -nt 1

# Check if iqtree2 ran successfully and the output file was created
if [ -f "$output_file" ]; then
    echo "$OG2" >> finished_processing.txt
else
    echo "$OG2" >> error_processing.txt
fi

}
```

```
export -f run_C60

parallel --jobs $CPUS run_C60 {} :::: gene_trees.filelist
```

7.5 Tree statistics and submatrices

```
# Language: Bash
phykit () {

SUPERMATRIX=$1
file=$2
file2=`echo $file | awk -F'[/|.]' '{print $2".tree"}'`

echo "$SUPERMATRIX $file $file2"

tax_occ=`grep -E "^>[0-9]+|[0-9]+$" ${SUPERMATRIX}_data/$file | awk -F"|" '{print $1}' | sort |
uniq | wc -l`
seqcount=`grep ">" ${SUPERMATRIX}_data/$file | wc -l`
echo -e "$file\t$tax_occ\t$seqcount" >> phykit/tax_occ.tsv

pk_rcv ${SUPERMATRIX}_data/$file | awk -F'\t' -vOFS='\t' -valn="$file" '{print aln,$0}' >>
phykit/pk_rcv.tsv

pk_pis ${SUPERMATRIX}_data/$file | awk -F'\t' -vOFS='\t' -valn="$file" '{print aln,$0}' >>
phykit/pk_pis.tsv

pk_evo_rate ${SUPERMATRIX}_trees/${file2} | awk -F'\t' -vOFS='\t' -valn="$file" '{print
aln,$0}' >> phykit/pk_evo_rate.tsv

pk_aln_len_no_gaps ${SUPERMATRIX}_data/$file | awk -F'\t' -vOFS='\t' -valn="$file" '{print
aln,$1,$2,$3}' >> phykit/pk_aln_len_no_gaps.tsv

pk_lb_score ${SUPERMATRIX}_trees/${file2} | awk -F': ' -vOFS='\t' -valn="$file"
'{if($1=="standard deviation") print aln,$2}' >> phykit/pk_dslb_score.tsv

pk_lb_score ${SUPERMATRIX}_trees/${file2} | awk -F': ' -vOFS='\t' -valn="$file"
'{if($1=="mean") print aln,$2}' >> phykit/pk_alb_score.tsv

pk_saturation -a ${SUPERMATRIX}_data/$file -t ${SUPERMATRIX}_trees/${file2} | awk -F'\t'
-vOFS='\t' -valn="$file" '{print aln,$0}' >> phykit/pk_saturation.tsv

pk_tness ${SUPERMATRIX}_trees/${file2} | awk -F'\t' -vOFS='\t' -valn="$file" '{print aln,$0}'
>> phykit/pk_tness.tsv

pk_tor -a ${SUPERMATRIX}_data/$file -t ${SUPERMATRIX}_trees/${file2} | awk -F'\t' -vOFS='\t'
-valn="$file" '{print aln,$0}' >> phykit/pk_tor.tsv

pk_bss ${SUPERMATRIX}_trees/${file2} | awk -F': ' -vOFS='\t' -valn="$file" '{if($1=="mean")
print aln,$2}' >> phykit/pk_abs.tsv

}

export -f phykit
```

```
mkdir -p phykit


parallel --jobs $CPUS phykit $SUPERMATRIX {1} :::: ${SUPERMATRIX}.filelist
```

8 Computing species trees

8.1 Concatenation approach

8.1.1 Edge-proportional partitioned model

```
# Language: Bash

iqtree2 -s ${SUPERMATRIX}_data -p "${DATA_DIR}/gene_alignments/${SUPERMATRIX}.partition.nex"
--seqtype AA --prefix EDGEPROP.${SUPERMATRIX} -ntmax $CPUS -nt $CPUS -B 1000 --alrt 1000
```

8.1.2 Site-heterogeneous model

```
# Language: Bash
CPUS=$1
SUPERMATRIX=$2
TOPO=SITEHET.${SUPERMATRIX}

TMPDIR="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}"
mkdir -p "$TMPDIR"
cd "$TMPDIR"

mamba activate catfasta2phyml
AMAS.py concat -p ${SUPERMATRIX}.partitions.nex.txt -t ${SUPERMATRIX}.concatenated.fa -u fasta
-y nexus -i ${SUPERMATRIX}_data/* -f fasta -d aa
mamba deactivate

iqtree2 -s ${SUPERMATRIX}.concatenated.fa -m LG+C60+F+G -ft EDGEPROP.${SUPERMATRIX}.treefile
--seqtype AA --prefix ${TOPO} -ntmax $CPUS -nt $CPUS -mem 600G -n 0 -redo

iqtree2 -s ${SUPERMATRIX}.concatenated.fa -m LG+C60+F+G -fs ${TOPO}.sitefreq --seqtype AA
--prefix ${TOPO} -ntmax $CPUS -nt $CPUS -B 1000 -alrt 1000 --undo
```

8.2 Reconciliation approach

```
# Language: Bash

mpiexec -np $CPUS asteroid --input-gene-trees ${SUPERMATRIX}.genetrees.nx
--gene-species-mapping ${SUPERMATRIX}.genetrees.map.txt --min-bl 0.01 --bs-replicates 100
--prefix ${SUPERMATRIX}.asteroid
```

8.3 Computing constrained trees

```
# Language: Bash
CPUS=$1
TOPO=$2

TMPDIR="${SCRATCH_DIR}/tree_building/constrained"
mkdir -p "$TMPDIR"
cd "$TMPDIR"

### Site homogeneous ###

iqtree2 -S ${SUPERMATRIX}_data -p "${DATA_DIR}/gene_alignments/${SUPERMATRIX}.partition.nex"
--seqtype AA --prefix EDGEPROP.${SUPERMATRIX}.constrained_to_${TOPO} -ntmax $CPUS -nt $CPUS -te
"${DATA_DIR}/trees/mycladetopo.${TOPO}.tree"

### Site heterogeneous ###

iqtree2 -s ${SUPERMATRIX}.concatenated.fa -m LG+C60+F+G -ft EDGEPROP.${SUPERMATRIX}.treefile
--seqtype AA --prefix SITEHET.${SUPERMATRIX}.constrained_to_${TOPO} -ntmax $CPUS -nt $CPUS -mem
600G -n 0 -te "${DATA_DIR}/trees/mycladetopo.${TOPO}.tree"

iqtree2 -s ${SUPERMATRIX}.concatenated.fa -m LG+C60+F+G -ft EDGEPROP.${SUPERMATRIX}.treefile
--seqtype AA --prefix SITEHET.${SUPERMATRIX}.constrained_to_${TOPO} -ntmax $CPUS -nt $CPUS -B
1000 -alrt 1000 --scfl 100 --undo

####################
treels () {
cat "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/EDGEPROP.${SUPERMATRIX}.treefile" >
EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.treels

for TOPO in topoM1d topoM2d topoM3d topoM4d topoM5d \
            topoC1d topoC2d topoC3d topoC4d \
            topoS1d topoS2d topoS3d topoS4d topoS5d

do
cat EDGEPROP.${SUPERMATRIX}.constrained_to_${TOPO}.treefile >>
EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.treels

done

cp EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.treels
$MYDIR/EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.treels
}

#########
post_processing () {
grep -B 5 "BIC" "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/EDGEPROP.${SUPERMATRIX}.iqtree" |
awk -F'\t' -vOFS='\t' '{print "SOG10",$0}' >
EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.tree_scores


for TOPO in topoM1d topoM2d topoM3d topoM4d topoM5d \
            topoC1d topoC2d topoC3d topoC4d \
            topoS1d topoS2d topoS3d topoS4d topoS5d
do
```

```
grep -B 5 "BIC" EDGEPROP.${SUPERMATRIX}.constrained_to_${TOPO}.iqtree | awk -F'\t' -vOFS='\t'
-vtopo="$TOPO" '{print topo,$0}' >> EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.tree_scores

done

cp EDGEPROP.${SUPERMATRIX}.constrained_to_topo_d.tree_scores $MYDIR/.
}
```

9 Assessing node support

9.1 For species trees

```
# Language: Bash

CPUS=$1

SUPERMATRIX=SOG10
MODEL=EDGEPROP

GENE_TREES="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/${SUPERMATRIX}.genetrees.renamed.nx"

cd "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}"

#get qCF for SOG
astral4 --root "87" -C -t $CPUS -u 3 -q ${MODEL}.${SUPERMATRIX}.treefile -i ${GENE_TREES} -t 2
-o ${MODEL}.${SUPERMATRIX}.astral_species_annotated.tree 2>
${MODEL}.${SUPERMATRIX}.astral_species_annotated.log

#Get sCF
iqtree2 -te ${MODEL}.${SUPERMATRIX}.astral_species_annotated.tree --scfl 100 -s
"${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/${SUPERMATRIX}_data" -p
"${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/${SUPERMATRIX}.partition.nex" --prefix
${MODEL}.${SUPERMATRIX}.scfl -T $CPUS

#Get gCF
iqtree2 -te ${MODEL}.${SUPERMATRIX}.astral_species_annotated.tree --gcf ${GENE_TREES} --prefix
${MODEL}.${SUPERMATRIX}.gcf -T $CPUS

#Get branch lengths in coalescent units
iqtree2 -te ${MODEL}.${SUPERMATRIX}.astral_species_annotated.tree -blfix -p
"${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/${SUPERMATRIX}.partition.nex" --scfl 1 --prefix
${MODEL}.${SUPERMATRIX}.coalescent_bl -T $CPUS
```

```
# Language: Bash

CPUS=$1

SUPERMATRIX=OG75
MODEL=MAT

GENE_TREES="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/${SUPERMATRIX}.genetrees.renamed.nx"
```

```
cd "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}"

#get qCF for OG

GENE_TREES="${DATA_DIR}/trees/.${SUPERMATRIX}.genetrees.nx"
GENE_MAP="${DATA_DIR}/trees/.${SUPERMATRIX}.genetrees.seqlist.txt"

#root "22" for A31 dataset
astral-pro --root "87" -R -t $CPUS -u 3 --genelength 500 -o
${MODEL}.${SUPERMATRIX}.astral-pro3_species_annotated.tree -i ${GENE_TREES} -a ${GENE_MAP} 2>
${MODEL}.${SUPERMATRIX}.astral-pro3_species_annotated.log
```

9.2 For constrained trees

The commands below use only the constrained hypotheses retained in the final manuscript and Supplementary Information: Topo A1–A4, M1–M5, C1–C4, and S1–S5. Repository filenames use these final manuscript identifiers.

```
# Language: Bash

cd "${SCRATCH_DIR}/tree_building"

MYDIR="${DATA_DIR}/trees/key_results_topo_d"


############# getting tree scores summary table ######################
#####################################################################

rm SITEHET2.constrained_to_topo_d.tree_scores
rm EDGEPROP.constrained_to_topo_d.tree_scores

for SUPERMATRIX in SOG10_tax_occ L42_synt L38_synt A31_fullcov
do
TOPO=species_tree
grep -B 5 "BIC" "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/SITEHET2.${SUPERMATRIX}.iqtree" |
awk -F'\t' -vOFS='\t' -vSUP="$SUPERMATRIX" -vTOPO="$TOPO" '{print SUP".C60",TOPO,$0}' >>
SITEHET2.constrained_to_topo_d.tree_scores


for TOPO in topoA1d topoA2d topoA3d topoA4d \
            topoM1d topoM2d topoM3d topoM4d topoM5d \
            topoC1d topoC2d topoC3d topoC4d \
            topoS1d topoS2d topoS3d topoS4d topoS5d
do

grep -B 5 "BIC"
"${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/SITEHET2.${SUPERMATRIX}.constrained_to_${TOPO}.iqtree"
| awk -F'\t' -vOFS='\t' -vSUP="$SUPERMATRIX" -vTOPO="$TOPO" '{print SUP".C60",TOPO,$0}' >>
SITEHET2.constrained_to_topo_d.tree_scores

done

done

for SUPERMATRIX in SOG10_all SOG10_tax_occ L42_synt L38_synt A31_fullcov
do
```

```
TOPO=species_tree
grep -B 5 "BIC" "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/EDGEPROP.${SUPERMATRIX}.iqtree" |
awk -F'\t' -vOFS='\t' -vSUP="$SUPERMATRIX" -vTOPO="$TOPO" '{print SUP".MAT",TOPO,$0}' >>
EDGEPROP.constrained_to_topo_d.tree_scores


for TOPO in topoA1d topoA2d topoA3d topoA4d \
            topoM1d topoM2d topoM3d topoM4d topoM5d \
            topoC1d topoC2d topoC3d topoC4d \
            topoS1d topoS2d topoS3d topoS4d topoS5d
do

grep -B 5 "BIC"
"${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/EDGEPROP.${SUPERMATRIX}.constrained_to_${TOPO}.iqtree"
| awk -F'\t' -vOFS='\t' -vSUP="$SUPERMATRIX" -vTOPO="$TOPO" '{print SUP".MAT",TOPO,$0}' >>
EDGEPROP.constrained_to_topo_d.tree_scores

done
done

cat SITEHET2.constrained_to_topo_d.tree_scores EDGEPROP.constrained_to_topo_d.tree_scores >
constrained_to_topo_d.tree_scores

awk -v OFS='\t' '
BEGIN {
  print "SUPERMATRIX","topo","LL","LL_se","LL_unconstrained","n_free_params","AIC","AICc","BIC"
}
{
  sup=$1; topo=$2; key=sup SUBSEP topo
  if (!(key in seen)) { seen[key]=++n; order[n]=key }
  rest=""
  for (i=3;i<=NF;i++) rest = rest (i==3?"":" ") $i

  if (match(rest, /Log-likelihood of the
tree:[[:space:]]*(-?[0-9.]+)[[:space:]]*\(s\.e\.[[:space:]]*([0-9.]+)\)/, m)) { LL[key]=m[1];
LLse[key]=m[2] }
  else if (match(rest, /Unconstrained log-likelihood \(without tree\):[[:space:]]*(-?[0-9.]+)/,
m)) { LLu[key]=m[1] }
  else if (match(rest, /Number of free parameters .*:[[:space:]]*([0-9]+)/, m)) {
npar[key]=m[1] }
  else if (match(rest, /Akaike information criterion \(AIC\) score:[[:space:]]*(-?[0-9.]+)/,
m)) { AIC[key]=m[1] }
  else if (match(rest, /Corrected Akaike information criterion \(AICc\)
score:[[:space:]]*(-?[0-9.]+)/, m)) { AICc[key]=m[1] }
  else if (match(rest, /Bayesian information criterion \(BIC\) score:[[:space:]]*(-?[0-9.]+)/,
m)) { BIC[key]=m[1] }
}
END {
  for (i=1;i<=n;i++) {
    k=order[i]
    split(k, a, SUBSEP)
    print a[1], a[2], LL[k], LLse[k], LLu[k], npar[k], AIC[k], AICc[k], BIC[k]
  }
}
' constrained_to_topo_d.tree_scores > constrained_to_topo_d.tree_scores.tsv
```

```
cp constrained_to_topo_d.tree_scores.tsv $MYDIR/.
```

9.3 PhyTop

```
# Language: Bash
mamba activate phytop

A144_CLADE_FILE="${DATA_DIR}/trees/A144_clades.csv"
L38_CLADE_FILE="${DATA_DIR}/trees/L38_clades.csv"
TOPO=$1       # e.g., topo29
OUTGROUP_ID=$2

for SUPERMATRIX in SOG10 SOG10_C60 L38_synt L38_synt_C60; do
  case "$SUPERMATRIX" in
    SOG10*) CLADE_FILE="$A144_CLADE_FILE" ;;
    L38*)   CLADE_FILE="$L38_CLADE_FILE" ;;
  esac

  cd "${SCRATCH_DIR}/tree_building/${SUPERMATRIX}"
  NEWICK=${TOPO}.${SUPERMATRIX}.astral4_species_annotated.tree

  phytop -g "$NEWICK" -outgroup "$OUTGROUP_ID" -clades "$CLADE_FILE" -collapse
done
```

10 Incomplete lineage sorting and introgression

The QuIBL batch analysis and the calculation of D and f4 statistics involve substantial custom Python workflows. Cleaned representative implementations are provided as `Scripts/ILS_introgression/QuIBL_wrapper.ipynb` and `Scripts/ILS_introgression/Dstat_from_alignment.ipynb`, respectively. The QuIBL notebook retains one representative deep-branch analysis, and the D/f4 notebook retains the L42 implementation; the same workflows were applied to the other reported branches or matrices by changing their configuration.

11 Hypothesis testing

11.1 SH and AU tests

The complete parameterized SH/AU test wrapper, including selection of the highest-likelihood reference topology and parsing of the IQ-TREE report, is provided as `Scripts/Hypothesis_testing/constrained_topology_tests.sh`. For example, the edge-proportional SOG10_all analysis of Topo A1–A4 was invoked as:

```
# Language: Bash
bash "${REPOSITORY_ROOT}/Scripts/Hypothesis_testing/constrained_topology_tests.sh" \
  A SOG10_all MAT 1234
```

11.2 Likelihood mapping

```
# Language: Bash
CPUS=$1
SUPERMATRIX=$2
myvar=$3

TMPDIR="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}"
mkdir -p "$TMPDIR"
cd "$TMPDIR"

var=${myvar}_quartets
var_low="${myvar,,}"

iqtree2 -S ${SUPERMATRIX}_data -p ${SUPERMATRIX}.partition.nex --seqtype AA --prefix
EDGEPROP.quartet.quartet_map_${var_low}.${SUPERMATRIX}_${var} -lmclust
"${DATA_DIR}/trees/quartet_map_${var_low}.nex" -ntmax $CPUS -nt $CPUS -n 0 --undo

iqtree2 -s ${SUPERMATRIX}.concatenated.fa -m LG+C60+F+G -fs SITEHET2.${SUPERMATRIX}.sitefreq
--seqtype AA -te SITEHET2.${SUPERMATRIX}.treefile --prefix
SITEHET2.quartet.quartet_map_${var_low}.${SUPERMATRIX}_${var} -lmclust
"${DATA_DIR}/trees/quartet_map_${var_low}.nex" -ntmax $CPUS -nt $CPUS -n 0
```

Parsing and plotting of the Oweniidae and Sipuncula results are documented in the cleaned standalone notebook `Scripts/Hypothesis_testing/Parse_QuartetLikelihoodMapping_results.ipynb`. Other exploratory focal-clade analyses were omitted.

11.3 DeltaL analysis

```
# Language: Bash
CPUS=$1
SUPERMATRIX=$2

TMPDIR="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}"
mkdir -p "$TMPDIR"
cd "$TMPDIR"


REF_ALN="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/${SUPERMATRIX}.concatenated.fa"
REF_SITEFREQ="${SCRATCH_DIR}/tree_building/${SUPERMATRIX}/SITEHET2.${SUPERMATRIX}.sitefreq"

for TOPO in topoA1d topoA2d topoA3d topoA4d

do

iqtree2 -S ${SUPERMATRIX}_data -p ${SUPERMATRIX}.partition.nex --seqtype AA --prefix
EDGEPROP.${SUPERMATRIX}.constrained_to_${TOPO} -ntmax $CPUS -nt $CPUS -te
EDGEPROP.${SUPERMATRIX}.constrained_to_${TOPO}.treefile -blfix -wsl --undo

iqtree2 -s $REF_ALN -m LG+C60+F+G -fs $REF_SITEFREQ --seqtype AA --prefix
SITEHET2.${SUPERMATRIX}.constrained_to_${TOPO} -ntmax $CPUS -nt $CPUS -te
SITEHET2.${SUPERMATRIX}.constrained_to_${TOPO}.treefile -blfix -wsl --undo

done
```

The remaining DeltaL workflow is provided in `Scripts/Hypothesis_testing/DeltaL_analysis.ipynb`. The notebook combines the site-wise likelihoods for Topo A1–A4, generates the likelihood-deficit figure, removes the most opinionated sites, and performs the reported gene-enrichment analysis.

12 Time calibration

12.1 Get calibrations

```r
# Language: R

# Get Skew-student distribution parameters for root node

library(CladeDate)
annelida<-t(matrix(c(c(5.145,5.21),c(5.06,5.145),c(5.053,5.097),c(5.16,5.21),c(5.06,5.705)),nrow=2))

Calib=clade.date(ages = annelida, n = 10000, plot=TRUE,method = "OLE", PDFfitting =
'skewstudent',KStest=T)
summary.clade.date(Calib)
```

12.2 Run time tree

```bash
# Language: Bash
# dry run to get the priors

CPUS=40
CALIB_VERSION=$1 #v5
ALIGNMENT=$2 #SOG10_tax_occ
TOPO=$3 #topo29
###

mkdir -p "${SCRATCH_DIR}/tree_building/time_tree/${CALIB_VERSION}_dryrun"
cd "${SCRATCH_DIR}/tree_building/time_tree/${CALIB_VERSION}_dryrun"

ln -s "${DATA_DIR}/gene_alignments/${ALIGNMENT}.fasta" .
ln -s "${DATA_DIR}/trees/${TOPO}.with_time_calibrations_${CALIB_VERSION}.phy" .

sed "s/\!\!\!ALIGNMENT\!\!\!/$ALIGNMENT/g; s/\!\!\!CALIB\!\!\!/$CALIB_VERSION/g;
s/\!\!\!TOPO\!\!\!/$TOPO/g" \
  "${SCRATCH_DIR}/tree_building/time_tree/dryrun.ctl" > "${TOPO}.${CALIB_VERSION}_dryrun.ctl"

mcmctree "${TOPO}.${CALIB_VERSION}_dryrun.ctl"
```

```bash
# Language: Bash
# Build hessian matrix

CPUS=40
SUPERMATRIX=$1 #SOG10_tax_occ
CALIB_VERSION=$2 #v14
TOPO=$3 #topo29
```

```
mkdir -p "${SCRATCH_DIR}/tree_building/time_tree/LG.${SUPERMATRIX}"
cd "${SCRATCH_DIR}/tree_building/time_tree/LG.${SUPERMATRIX}"

ln -s "${DATA_DIR}/trees/${TOPO}.with_time_calibrations_${CALIB_VERSION}.tree" .

iqtree3 -s ${SUPERMATRIX}.concatenated.renamed.fa --seqtype AA -m LG+I+G -te
${TOPO}.with_time_calibrations_${CALIB_VERSION}.tree --seqtype AA -ntmax $CPUS -nt $CPUS -mem
100G --dating mcmctree --mcmc-clock IND --prefix ${TOPO}.LG.${SUPERMATRIX} --redo-tree

mkdir -p "${SCRATCH_DIR}/tree_building/time_tree/SITEHET2.${SUPERMATRIX}"
cd "${SCRATCH_DIR}/tree_building/time_tree/SITEHET2.${SUPERMATRIX}"


ln -s "${DATA_DIR}/trees/${TOPO}.with_time_calibrations_${CALIB_VERSION}.tree" .

iqtree3 -s ${SUPERMATRIX}.concatenated.renamed.fa -m LG+C60+F+G -fs
SITEHET2.${SUPERMATRIX}.sitefreq -te ${TOPO}.with_time_calibrations_${CALIB_VERSION}.tree
--seqtype AA --prefix ${TOPO}.SITEHET2.${SUPERMATRIX} -ntmax $CPUS -nt $CPUS --dating mcmctree
--mcmc-clock IND --redo-tree
```

```
# Language: Bash
# run MCMC chains
# run_mcmc_parallel.sh v14 LG.SOG10_tax_occ topo29

CALIB_VERSION=$1 #v5
ALIGNMENT=$2 #SITEHET2.SOG10_tax_occ
TOPO=$3 #topo29

###

mkdir -p "${SCRATCH_DIR}/tree_building/time_tree/${ALIGNMENT}/${CALIB_VERSION}"
cd "${SCRATCH_DIR}/tree_building/time_tree/${ALIGNMENT}/${CALIB_VERSION}"

ln -s "${DATA_DIR}/trees/${TOPO}.with_time_calibrations_${CALIB_VERSION}.phy" .

run_MCMCTree_chain() {

RUNID=$1
CALIB_VERSION=$2
ALIGNMENT=$3
TOPO=$4

mkdir -p run_$RUNID

CTL_FILE="${SCRATCH_DIR}/tree_building/time_tree/${ALIGNMENT}/${CALIB_VERSION}/"
CTL_FILE+="run_${RUNID}/run_${RUNID}.${TOPO}_${CALIB_VERSION}.mcmctree.ctl"

sed "s/\!\!\!ALIGNMENT\!\!\!/$ALIGNMENT/g; s/\!\!\!CALIB\!\!\!/$CALIB_VERSION/g;
s/\!\!\!TOPO\!\!\!/$TOPO/g; s/\!\!\!RUNID\!\!\!/$RUNID/g" \
  "${SCRATCH_DIR}/tree_building/time_tree/template.mcmctree.ctl" \
  > "$CTL_FILE"

cd run_$RUNID

mcmctree "run_${RUNID}.${TOPO}_${CALIB_VERSION}.mcmctree.ctl" >
"run_${RUNID}.${TOPO}_${CALIB_VERSION}.log" 2>&1
```

```
}

export -f run_MCMCTree_chain

parallel -j 4 run_MCMCTree_chain {1} ${CALIB_VERSION} ${ALIGNMENT} ${TOPO} ::: {1..4}
```

12.3 Assess convergence and parse results

Chain diagnostics and the dry-run and posterior summaries are generated by the standalone `MCMCTree_postprocessing.py` script in `Scripts/Divergence_times/`. The final analyses used calibration version v14 and both protein-evolution models:

```
# Language: Bash
python "${REPOSITORY_ROOT}/Scripts/Divergence_times/MCMCTree_postprocessing.py" \
  --analysis-root "${ANALYSIS_ROOT}" \
  --topology topo29 \
  --calibration v14
```